



Instituto Politécnico Nacional

Escuela Superior de Cómputo



Programa académico / Plan de estudios

Ingeniería en Sistemas Computacionales / 2020

Unidad de aprendizaje

Desarrollo de aplicaciones móviles nativas

Práctica 2: Aplicación móvil básica para operaciones CRUD con un servicio REST

Objetivo: El propósito de esta práctica es que desarrollen una aplicación móvil que permita realizar operaciones CRUD (Crear, Leer, Actualizar, Borrar) sobre un servicio REST dockerizado. Además, deberán implementar un sistema de autenticación que permita el inicio de sesión y el registro de usuarios, asegurando que las contraseñas estén encriptadas y las sesiones sean seguras. La entrega se realiza en un repositorio público de GitHub con su respectivo archivo README.md.

Repositorio de ejemplo

Punto de partida: Para esta práctica se pone a su disposición un repositorio de ejemplo que pueden clonar o del cual pueden hacer un fork. Contiene un backend en Flask ya dockerizado y una aplicación Android en Kotlin con Jetpack Compose lista para conectarse a él.

1. Contenido del repositorio:

- Liga del repositorio: <https://github.com/gabrielhuav/Flask-Compose-Login-API>
 - **Docker-Flask/ORM:** API REST minimalista construida con Flask, que utiliza Flask-SQLAlchemy para la persistencia en SQLite y Flask-Bcrypt para el hash de contraseñas. No requiere una base de datos externa: el archivo de la base se crea automáticamente al iniciar. Expone los endpoints de verificación (GET a la raíz), registro (POST a /register) e inicio de sesión (POST a /login).
 - **Android/FlaskLogin:** proyecto base generado en Android Studio con Kotlin y Jetpack Compose (Material 3), con minSdk 24 y targetSdk 36. La lógica de conexión con la API está intencionalmente incompleta: es parte de lo que ustedes deben implementar.

2. Cómo utilizarlo:

- Clonar el repositorio, entrar a la carpeta Docker-Flask/ORM y levantar el servicio con el comando: `docker compose up --build`
- El servicio queda disponible en el puerto 5000 del equipo anfitrión. Verificar los endpoints con curl, Postman o Insomnia antes de conectar la aplicación móvil.

3. Alcance del ejemplo:

- El repositorio es únicamente un punto de partida, no la entrega. Sobre esa base deberán implementar las operaciones CRUD completas sobre un recurso a elección, las pantallas de inicio de sesión y registro, y la conexión real entre la aplicación y la API.
- Si parten del ejemplo, deberán indicarlo explícitamente en su README y describir qué archivos modificaron o agregaron. Entregar el ejemplo sin cambios se considera práctica no realizada.

Ejercicio 1: Configuración inicial y diseño de la aplicación

1. Diseño del Menú de Navegación:

- La aplicación debe incluir un menú desplegable o algún componente de navegación similar que permita a los usuarios acceder a las opciones de inicio de sesión y registro.
- Las opciones del menú deberán permitir realizar las siguientes acciones:
 - Inicio de Sesión
 - Registro de Usuario

- Operaciones CRUD
- 2. **Interfaz de Usuario:**
 - Diseñar una interfaz atractiva para móviles. Se recomienda el uso de frameworks de diseño responsivo o componentes nativos de Android.
 - Asegurarse de que la navegación sea intuitiva para el usuario final.
 - Manejar los estados de la interfaz: carga, error y sesión iniciada, de manera que el usuario siempre sepa qué está ocurriendo.

Ejercicio 2: Backend REST dockerizado

1. **Conceptos que deberán comprender y documentar:**
 - **Docker:** plataforma que empaqueta una aplicación junto con todo lo que necesita para ejecutarse (runtime, librerías, dependencias y configuración) en una unidad aislada llamada contenedor. A diferencia de una máquina virtual, comparte el núcleo del sistema operativo anfitrión, por lo que arranca en segundos. Su ventaja principal es la reproducibilidad: el proyecto se ejecuta igual en cualquier equipo.
 - **Imagen y contenedor:** la imagen es la plantilla inmutable a partir de la cual se crea el contenedor; el contenedor es la instancia en ejecución de esa imagen y es efímero, por lo que la información que deba conservarse se guarda en un volumen.
 - **Dockerfile:** archivo de texto con las instrucciones (FROM, WORKDIR, COPY, RUN, EXPOSE, CMD) que Docker ejecuta paso a paso para construir la imagen de su backend.
 - **docker-compose.yml:** archivo en formato YAML que describe la aplicación como un conjunto de servicios, con sus puertos, volúmenes, redes y variables de entorno, de manera que todo el entorno se levante o se detenga con un solo comando.
 - **Backend o servicio REST:** programa que corre del lado del servidor y expone la lógica de negocio mediante rutas accesibles por HTTP. Recibe peticiones con los verbos GET, POST, PUT y DELETE, valida la información, consulta o modifica la base de datos y responde en formato JSON con el código de estado correspondiente.
 - **ORM y base de datos:** el ORM (por ejemplo SQLAlchemy) permite manipular las tablas de la base de datos como objetos del lenguaje, sin escribir SQL directamente. En el ejemplo se emplea SQLite, que guarda toda la información en un archivo local.
2. **Operaciones CRUD:**
 - Utilizar el backend desarrollado en la Tarea Previa, partir del repositorio de ejemplo o construir uno nuevo. El servicio debe estar dockerizado con Docker Compose y exponer los endpoints necesarios para las operaciones CRUD sobre un recurso a elección (por ejemplo, productos, tareas o notas).
 - **POST:** Crear un nuevo registro.
 - **GET:** Leer uno o varios registros.
 - **PUT:** Actualizar un registro existente.
 - **DELETE:** Borrar un registro.
 - Las respuestas deben entregarse en formato JSON y utilizar los códigos de estado HTTP adecuados (200, 201, 400, 401, 404).
 - Publicar el servicio en el mismo repositorio público de GitHub que la aplicación Android, incluyendo el Dockerfile y el docker-compose.yml.
3. **Levantamiento del entorno:**
 - El backend debe poder ejecutarse desde cero, en una máquina que únicamente tenga Docker instalado, con el comando: `docker compose up --build`
 - En el README deberán documentar los pasos previos (clonar el repositorio, ubicarse en la carpeta del backend, configurar variables de entorno si aplica) y qué hace cada instrucción de su Dockerfile y de su docker-compose.yml.
 - Si por alguna razón técnica deciden levantar el proyecto de otra forma (un solo contenedor con `docker run`, un despliegue en la nube, una base de datos gestionada externa), deberán incluir en el README una sección de decisiones técnicas explicando qué alternativa eligieron, por qué fue necesaria, qué ventajas y desventajas tiene frente a Docker Compose y cuál es el procedimiento exacto y reproducible para levantar el servicio.
4. **Autenticación de Usuarios:**
 - Implementar un sistema de autenticación que permita:
 - El registro de usuarios con contraseñas encriptadas mediante una función de hash con sal (bcrypt, Argon2 o equivalente). En ningún caso deben almacenarse en texto plano.

- El inicio de sesión para usuarios existentes.
 - Se sugiere la implementación de sesiones seguras para manejar la autenticación.
 - **Nota:** no es obligatorio el uso de Spring Boot ni de Flask; pueden elegir el lenguaje y framework que prefieran, siempre que justifiquen la elección en el README.
- 5. **Sesiones Seguras:**
 - Implementar un mecanismo de manejo de sesiones (por ejemplo, un token firmado con tiempo de expiración) para que solo los usuarios autenticados puedan realizar las operaciones CRUD.
 - Las sesiones deben estar protegidas contra vulnerabilidades como el robo de sesión, y los endpoints protegidos deben responder con 401 cuando no exista una sesión válida.
 - Las credenciales, llaves y secretos no deben quedar escritos en el código ni subirse al repositorio: utilicen variables de entorno y publiquen únicamente un archivo de ejemplo con los nombres de las variables.

Ejercicio 3: Implementación del sistema de autenticación en la aplicación móvil

1. **Registro de Usuario y Login en la Aplicación Móvil:**
 - Desarrollar una pantalla de Inicio de Sesión y una de Registro de Usuario en la aplicación móvil.
 - Implementar la funcionalidad para conectar con el servicio REST y realizar las operaciones de autenticación (registro e inicio de sesión).
 - Las contraseñas enviadas desde la aplicación deben viajar protegidas y almacenarse siempre hasheadas del lado del servidor.
2. **Conexión a la API REST:**
 - La aplicación móvil deberá conectarse al servicio REST desarrollado en el ejercicio anterior, utilizando un cliente HTTP (por ejemplo, Retrofit con OkHttp) para realizar las solicitudes.
 - Declarar el permiso de INTERNET en el AndroidManifest.xml. Si la API se consume por HTTP sin TLS durante el desarrollo, será necesario habilitar el tráfico en texto claro (usesCleartextTraffic) dentro de la etiqueta application.
 - **Nota importante:** desde el emulador de Android, la dirección localhost apunta al propio emulador y no a su computadora. Para alcanzar el servicio que corre en contenedor deben utilizar la dirección 10.0.2.2 con el puerto publicado (por ejemplo, 10.0.2.2:5000). Si usan un dispositivo físico en la misma red, deberán emplear la dirección IP local de su equipo. La URL base y la forma de configurarla deben documentarse en el README.
3. **Pruebas en Emulador:**
 - Probar la aplicación móvil en un emulador de Android, asegurándose de que las operaciones CRUD y el sistema de autenticación funcionen correctamente.
 - Documentar el flujo completo con capturas de pantalla: registro, inicio de sesión, las cuatro operaciones CRUD y el manejo de credenciales incorrectas.

Entrega de la Práctica

Formato de entrega: La entrega se realiza exclusivamente en GitHub. No se entrega informe en Word ni en PDF: toda la documentación y las capturas de pantalla viven dentro del repositorio, en archivos con extensión .md. En Classroom únicamente se registra la liga del repositorio.

1. **Repositorio:**
 - Subir el código fuente de la aplicación móvil y del servicio REST al mismo repositorio público de GitHub, en carpetas separadas y con un archivo .gitignore adecuado a cada tecnología.
 - Asegurarse de que los endpoints y las sesiones sean seguras, y que las contraseñas estén encriptadas, mediante un proceso de QA documentado.
 - El historial de commits debe evidenciar trabajo incremental y, en el caso de equipos, la participación de todos los integrantes.
2. **Archivo README.md, con la siguiente estructura:**
 - **Portada:** nombre completo, número de boleta, grupo, nombre de la asignatura, nombre del profesor y fecha de entrega.
 - **Introducción:** explicación general de su código y de la lógica utilizada, incluyendo el stack elegido y su justificación.
 - **Desarrollo:** explicación de los conceptos del Ejercicio 2 con sus propias palabras, documentación de los endpoints (método, ruta, parámetros, ejemplo de petición y respuesta), instrucciones de instalación y ejecución con el comando docker compose up --build, y capturas de pantalla de la ejecución incrustadas en el archivo .md y almacenadas en una carpeta del repositorio (por ejemplo, docs/ o img/).
 - **Conclusiones:** principales retos y logros involucrados en el desarrollo de su práctica, así como las dificultades encontradas y cómo fueron resueltas.

- **Bibliografía:** lista completa de las fuentes consultadas siguiendo el formato APA, al final del documento.

Criterios de evaluación

1. **Funcionamiento:** la práctica se evaluará clonando el repositorio en un equipo limpio y siguiendo el README al pie de la letra. Si el entorno no levanta con lo documentado, la práctica se considerará no funcional.
2. **Cumplimiento de la especificación:** que estén implementadas las cuatro operaciones CRUD, el registro y el inicio de sesión.
3. **Seguridad:** contraseñas hasheadas, endpoints protegidos y ausencia de credenciales dentro del repositorio.
4. **Documentación:** claridad y completitud del README, incluidas las capturas de pantalla.
5. **Exposición:** se podrá solicitar una breve explicación oral del código y de las decisiones técnicas tomadas.

Notas adicionales

- **Plagio:** considere que el plagio es un delito. Cualquier evidencia de copia será sancionada con la anulación de ambos trabajos, el original y la copia. Se utilizarán herramientas para verificar la originalidad.
- **Uso del ejemplo:** clonar el repositorio de ejemplo es válido y recomendado, pero debe indicarse en el README y acompañarse de aportaciones propias claramente identificables en el historial de commits.
- **Entrega oportuna:** las prácticas entregadas con demora tendrán una reducción del 10% en la calificación por cada día de retraso, con un máximo de diez días.

Fecha de Entrega: La fecha límite para la entrega de esta práctica es el **viernes 18 de septiembre de 2026**. No se aceptarán entregas fuera de tiempo y forma.